

# Lección 8: Technical Debt

---

## Deuda Técnica Práctica

---

### Agenda

---

- ¿Qué es Deuda Técnica?
  - Tipos de Deuda
  - Medición Práctica
  - Prevención
- 

### ¿Qué es Deuda Técnica?

---

“Decisiones de diseño que son expedientes a corto plazo pero costosos a largo plazo” - Ward Cunningham

#### Como deuda financiera:

- Tomas prestado tiempo ahora
  - Pagas intereses después (mantenimiento más lento)
  - Puede quebrar el proyecto si no se maneja
- 

### Tipos de Deuda Técnica

---

#### Deuda Deliberada:

- Decisión consciente
- Trade-off (compensación) analizado
- Plan de pago definido

### 💥 Deuda Accidental:

- Por desconocimiento
- Falta de experiencia
- Sin intención

### 🔥 Deuda Crítica vs Manejable:

- Crítica: Bloquea features
- Manejable: Ralentiza desarrollo

---

## Herramientas de Línea de Comandos (1/2)

---

**grep:** Busca texto en archivos

```
grep "TODO" src/file.ts          # Busca "TODO" en un archivo
grep -r "TODO" src/              # -r = recursivo (toda la carpeta)
grep "TODO\|FIXME" src/file.ts  # \| = OR (busca TODO O FIXME)
```

**wc:** Word Count (cuenta líneas, palabras, caracteres)

```
wc -l file.txt                  # -l = cuenta líneas
grep "TODO" src/ | wc -l       # Cuenta cuántas líneas tienen "TODO"
```

---

## Herramientas de Línea de Comandos (2/2)

---

**Pipe (|):** Conecta comandos

```
grep "TODO" src/ | wc -l
# 1. grep busca "TODO" y retorna líneas que coinciden
# 2. | pasa esas líneas a wc
# 3. wc -l cuenta cuántas líneas son
```

---

## Medición Práctica

---

## Indicadores Simples:

```
# 1. TODO Count (conteo de TODOs)
grep -r "TODO\|FIXME\|HACK" src/ | wc -l

# 2. Code Smells (ESLint/SonarQube)
npm run lint | grep "warning\|error" | wc -l

# 3. Test Coverage Gaps (gaps en cobertura)
npm run test:coverage | grep "Uncovered"

# 4. Build Time Trend (tendencia de tiempo de build)
# Aumento sostenido indica complejidad creciente
```

---

## Cuantificación de Deuda

---

### Cálculo Simple:

Deuda = Esfuerzo para Arreglar / Esfuerzo Total

Ejemplo:

- 5 días para arreglar smells
- 20 días de sprint
- Deuda Ratio = 25%

Red Flags (señales de alerta):

- > 30%: Muy alta
  - > 50%: Crítica
- 

## Estrategia: Boy Scout Rule

---

“Deja el código mejor de como lo encontraste”

```
// Cada vez que tocas código, méjoralo un poco
function addFeature() {
  // Mientras añades feature...
  // ...limpia código relacionado
  // ...arregla smells obvios
  // ...mejora nombres
```

}

**Resultado:** Deuda disminuye naturalmente

---

## Estrategia: Debt Sprints

---

**Sprint dedicado a pagar deuda:**

Sprint Deuda (1 semana cada trimestre):

- 0 features nuevas
- Solo refactoring y limpieza
- Pago de deuda prioritaria
- Mejora de tests

ROI (retorno sobre inversión): Velocity (velocidad de desarrollo) aumenta 2

---

## Estrategia: 80/20 Rule (Regla 80/20)

---

**80% features, 20% deuda:**

- Cada sprint: dedicar 20% a pagar deuda
  - Previene acumulación
  - Mantiene velocity (velocidad de desarrollo) estable
  - Sostenible a largo plazo
- 

## Prevención

---

**1. Definition of Done** (definición de terminado):

- Tests pasando
- Código revisado
- No nuevos TODO sin plan
- Documentación actualizada

**2. Quality Gates** (compuertas de calidad):

- Checks automatizados (Husky)
- Umbrales de coverage
- ESLint pasando

### 3. Regular Reviews (revisiones regulares):

- Revisión semanal de deuda
- Planificación mensual de debt sprint

---

## Comunicación con Stakeholders

---

**✗ Jerga técnica:** “Tenemos alta complejidad ciclomática y code smells”

**✓ Impacto en el negocio:** “Las nuevas features tardan el doble debido a deuda técnica. Invertir 1 semana ahora ahorra 4 semanas en el próximo trimestre.”

---

## Ejercicio 1: Resolver Deuda - Input Validation

---

### Prompt:

Actúa como un desarrollador que paga deuda técnica documentada con TODOs.

CONTEXT0: Technical Debt (deuda técnica) son decisiones de diseño expedient a corto plazo pero costosas a largo plazo (Ward Cunningham). TODOs document deuda deliberada con plan de pago. Boy Scout Rule: **"Deja el código mejor de como lo encontraste"**. Resolver TODOs antes que bloqueen features previene q deuda se vuelva crítica. Input validation (validación de entradas) es deuda común: código funciona con datos válidos pero falla con datos inesperados.

TAREA: Implementa input validation en calculateSubtotal para resolver TODO.

#### UBICACIÓN DEL TODO:

- Archivo: src/shared/utils/calculateSubtotal.ts
- Buscar comentario: // TODO: Add input validation

#### VALIDATION REQUIREMENTS:

- Si items es null o undefined → lanzar Error('Items array is required')
- Si algún item.price < 0 → lanzar Error('Price cannot be negative')
- Si algún item.quantity < 0 → lanzar Error('Quantity cannot be negative')
- Mantener comportamiento actual para casos válidos (reduce sum)

#### IMPLEMENTACIÓN:

1. Agregar validación de items null/undefined al inicio
2. Usar forEach para validar cada item antes del reduce
3. Lanzar errores descriptivos con throw new Error()
4. Mantener función pura (sin side effects)

#### TEST REQUIREMENTS:

- Framework: Vitest
- Archivo: src/shared/utils/calculateSubtotal.test.ts
- Test 1: items null → debe lanzar error
- Test 2: price negativo → debe lanzar error
- Test 3: quantity negativo → debe lanzar error
- Test 4: datos válidos → debe retornar subtotal correcto
- Usar: expect(() => calculateSubtotal(null)).toThrow('Items array is requi

VALIDACIÓN: ejecuta pnpm test calculateSubtotal → todos deben PASAR 

**Aprende:** Resolver deuda técnica documentada mejora robustez

y previene bugs en producción

---

## Ejercicio 2: Resolver Deuda - Extract Button Component

---

#### Prompt:

Actúa como un desarrollador aplicando DRY Principle (Don't Repeat Yourself)

CONTEXT0: Duplicate Code (código duplicado) es code smell que genera deuda técnica. Cada duplicación aumenta esfuerzo de mantenimiento: cambio en 1 lu requiere cambiar N lugares. Extract Component refactoring elimina duplicaci creando componente reutilizable en /shared/ (usado por 2+ features). Scope código usado por 1 feature → local, código usado por 2+ features → shared. React components reutilizables usan props para configuración flexible.

TAREA: Extrae botón duplicado a componente reutilizable en shared.

#### UBICACIÓN DEL TODO:

- Archivo: src/features/product-catalog/components/ProductCard.tsx
- Buscar comentario: // TODO: Extract button to shared component

#### COMPONENT SPECIFICATIONS:

- Nombre: Button
- Ubicación: src/shared/components/Button.tsx (usado por 2+ features)
- Props interface:
  - children: React.ReactNode (texto del botón)
  - onClick: () => void (handler de click)
  - disabled?: boolean (opcional, default false)
  - variant?: 'primary' | 'secondary' (opcional, default 'primary')

#### ESTILOS (Tailwind CSS):

- Base: 'py-2 px-4 rounded-lg font-medium transition-colors'
- Primary variant: 'bg-indigo-600 hover:bg-indigo-700 text-white'
- Secondary variant: 'bg-gray-600 hover:bg-gray-700 text-white'
- Combinar con template literals: className={` \${baseStyles} \${variantStyles}

#### REFACTORING STEPS:

1. Crear Button component en src/shared/components/Button.tsx
2. Refactorizar ProductCard para usar <Button variant="primary">
3. Refactorizar CartSummary para usar <Button variant="secondary">
4. Eliminar código de botón duplicado de ambos componentes

#### VALIDACIÓN:

- npm test → todos los tests deben PASAR 
- npm run build → build debe completar sin errores 

**Aprende:** Extract Component elimina duplicación y

centraliza cambios futuros (DRY Principle)

---

## Puntos Clave

---

1. **La deuda es normal:** Todos los proyectos tienen deuda
2. **Mídela:** Cuantifica para justificar inversión
3. **Págala:** Boy Scout Rule + Debt Sprints
4. **Prevénla:** Quality Gates + Definition of Done
5. **Comunícala:** Lenguaje de negocio, no jerga técnica
6. **Balancea:** Regla 80/20 mantiene velocity sostenible